

Лабораторная работа №7

Введение в работу с данными

Клюкин Михаил Александрович

Содержание

1	Введение	6
1.1	Цели и задачи	6
2	Выполнение лабораторной работы	7
2.1	Julia для науки о данных	7
2.2	Считывание данных	7
2.3	Запись данных в файл	11
2.4	Словари	13
2.5	DataFrames	15
2.6	RDatasets	17
2.7	Работа с переменными отсутствующего типа (Missing Values) . . .	19
2.8	FileIO	22
2.9	Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних	22
2.10	Кластеризация данных. Метод k ближайших соседей	29
2.11	Обработка данных. Метод главных компонент	31
2.12	Обработка данных. Линейная регрессия	32
2.13	Самостоятельное выполнение	36
3	Вывод	44
	Список литературы	45

Список иллюстраций

2.1	Установка пакетов	8
2.2	Считывание данных и запись в структуру	9
2.3	Пример	9
2.4	Поиск “julia” со строчной буквы	10
2.5	Изменение исходной функции	10
2.6	Построчное считывание данных	11
2.7	Запись данных в файл	11
2.8	Пример с указанием типа данных и разделителем данных	12
2.9	Проверка корректности считывания созданного текстового файла	13
2.10	Инициализация словаря	14
2.11	Инициализация пустого словаря	14
2.12	Заполнение словаря данными	14
2.13	Пример работы словаря	15
2.14	Пример создания структуры DataFrame	16
2.15	Пример создания структуры DataFrame	17
2.16	Работа с пакетом RDatasets	18
2.17	Получение основных статических сведений о каждом столбце в наборе данных	19
2.18	Использование “отсутствующего” типа. Операция сложения числа и переменной с отсутствующим типом	20
2.19	Пример работы с данными, среди которых есть данные с отсутствующим типом	20
2.20	Игнорирование отсутствующего типа	21
2.21	Формирование таблиц данных и их объединение в один фрейм	21
2.22	Подключение пакетов. Загрузка изображения	22
2.23	Загрузка данных	23
2.24	Построение графика цен на недвижимость в зависимости от площади	24
2.25	Построение графика без “артефактов”	25
2.26	Среднее значение	26
2.27	Построение графика с кластерами разных цветов	27
2.28	Построение графика с кластерами разных цветов	28
2.29	Построение графика с кластерами разных цветов по почтовому индексу	29
2.30	Отображение на графике соседей выбранного объекта недвижимости	30
2.31	Определение районов соседних домов	31

2.32 Попытка уменьшения размера данных о цене и площади из набора данных домов	32
2.33 Исходные данные	33
2.34 Применение функции для построения графика	34
2.35 Генерация данных. Определние времени соответствия	35
2.36 Сравнение	36
2.37 Кластеризация	37
2.38 Кластеризация	37
2.39 Кластеризация	38
2.40 Регрессия	39
2.41 Регрессия	39
2.42 Регрессия	40
2.43 Модель ценообразования биномиальных опционов	41
2.44 Модель ценообразования биномиальных опционов	41
2.45 Модель ценообразования биномиальных опционов	42
2.46 Модель ценообразования биномиальных опционов	42
2.47 Модель ценообразования биномиальных опционов	43
2.48 Модель ценообразования биномиальных опционов	43

Список таблиц

1 Введение

1.1 Цели и задачи

Цель работы

Основной целью работы является специализированных пакетов Julia для обработки данных[julialang?].

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы[juliadoc?].

2 Выполнение лабораторной работы

2.1 Julia для науки о данных

В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python.

2.2 Считывание данных

Перед тем, как начать проводить какие-либо операции над данными, необходимо их откуда-то считать и возможно сохранить в определённой структуре.

Довольно часто данные для обработки содержатся в csv-файле, имеющим текстовый формат, в котором данные в строке разделены, например, запятыми, и соответствуют ячейкам таблицы, а строки данных соответствуют строкам таблицы. Также данные могут быть представлены в виде фреймов или множеств.

В Julia для работы с такого рода структурами данных используют пакеты CSV, DataFrames, RDatasets, FileIO (рис. 2.1):

```
# Обновление окружения:
using Pkg
Pkg.update
# Установка пакетов:
using Pkg
for p in ["CSV", "DataFrames", "RDatasets", "FileIO"]
Pkg.add(p)
end
using CSV, DataFrames, DelimitedFiles
```

Рис. 2.1: Установка пакетов

Предположим, что у нас в рабочем каталоге с проектом есть файл с данными `programminglanguages.csv`, содержащий перечень языков программирования и год их создания. Тогда для заполнения массива данными для последующей обработки требуется считать данные из исходного файла и записать их в соответствующую структуру (рис. 2.2):


```
# Считывание данных и их запись в структуру:
P = CSV.File("programminglanguages.csv") |> DataFrame
```

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58
9	1959	FACT
10	1959	ALGOL 60

```
# Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

language_created_year (generic function with 1 method)
```

```
# Пример вызова функции и определение даты создания языка Python:
language_created_year(P, "Python")

1991
```

Рис. 2.2: Считывание данных и запись в структуру

Пример для Julia (рис. 2.3):

```
# Пример вызова функции и определение даты создания языка Julia:
language_created_year(P, "Julia")

2012
```

Рис. 2.3: Пример

В следующем примере при вызове функции, в качестве аргумента которой

указано слово `julia`, написанное со строчной буквы (рис. 2.4):

```
language_created_year(P, "julia")

MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)
The function `getindex` exists, but no method is defined for this combination of argument
types.

Closest candidates are:
  getindex(::DataFrame, ::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/dataframe/dataframe.jl:542
  getindex(::DataFrame, ::typeof(!), ::Union{Signed, Unsigned})
    @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/dataframe/dataframe.jl:548
  getindex(::DataFrame, ::InvertedIndex, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/dataframe/dataframe.jl:538
  ...

Stacktrace:
 [1] language_created_year(P::DataFrame, language::String)
    @ Main ./In[3]:4
 [2] top-level scope
    @ In[6]:1
 [3] eval(m::Module, e::Any)
    @ Core ./boot.jl:489
```

Рис. 2.4: Поиск “julia” со строчной буквы

Для того, чтобы убрать в функции зависимость данных от регистра, необходимо изменить исходную функцию следующим образом (рис. 2.5):

```
# Функция определения по названию языка программирования
# года его создания (без учёта регистра):
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
    return P[loc,1]
end
```

```
language_created_year_v2 (generic function with 1 method)
```

```
# Пример вызова функции и определение даты создания языка julia:
language_created_year_v2(P, "julia")
```

```
2012
```

Рис. 2.5: Изменение исходной функции

Можно считывать данные построчно, с элементами, разделёнными заданным разделителем (рис. 2.6):

```
# Построчное считывание данных с указанием разделителя:  
Tx = readlm("programminglanguages.csv", ',')
```

```
74x2 Matrix{Any}:  
  "year"  "language"  
1951      "Regional Assembly Language"  
1952      "Autocode"  
1954      "IPL"  
1955      "FLOW-MATIC"  
1957      "FORTRAN"  
1957      "COMTRAN"  
1958      "LISP"  
1958      "ALGOL 58"  
1959      "FACT"  
1959      "COBOL "  
1959      "RPG"  
1962      "APL"  
  ⋮  
2003      "Scala"  
2005      "F#"  
2006      "PowerShell"  
2007      "Clojure"  
2009      "Go"  
2010      "Rust"  
2011      "Dart"  
2011      "Kotlin"  
2011      "Red"  
2011      "Elixir"  
2012      "Julia"  
2014      "Swift"
```

Рис. 2.6: Построчное считывание данных

2.3 Запись данных в файл

Предположим, что требуется записать имеющиеся данные в файл. Для записи данных в формате CSV можно воспользоваться следующим вызовом (рис. 2.7):

```
# Запись данных в CSV-файл:  
CSV.write("programming_languages_data2.csv", P)  
  
"programming_languages_data2.csv"
```

Рис. 2.7: Запись данных в файл

Можно задать тип файла и разделитель данных (рис. 2.8):

```
# Пример записи данных в текстовый файл с разделителем ',':  
writedlm("programming_languages_data.txt", Tx, ',')  
# Пример записи данных в текстовый файл с разделителем '-':  
writedlm("programming_languages_data2.txt", Tx, '-')
```

Рис. 2.8: Пример с указанием типа данных и разделителем данных

Можно проверить, используя `readdlm`, корректность считывания созданного текстового файла (рис. 2.9):

```
# Построчное считывание данных с указанием разделителя:  
P_new_delim = readrlm("programming_languages_data2.txt", '-')
```

```
74x2 Matrix{Any}:  
  "year"  "language"  
1951      "Regional Assembly Language"  
1952      "Autocode"  
1954      "IPL"  
1955      "FLOW-MATIC"  
1957      "FORTRAN"  
1957      "COMTRAN"  
1958      "LISP"  
1958      "ALGOL 58"  
1959      "FACT"  
1959      "COBOL"  
1959      "RPG"  
1962      "APL"  
  ⋮  
2003      "Scala"  
2005      "F#"  
2006      "PowerShell"  
2007      "Clojure"  
2009      "Go"  
2010      "Rust"  
2011      "Dart"  
2011      "Kotlin"  
2011      "Red"  
2011      "Elixir"  
2012      "Julia"  
2014      "Swift"
```

Рис. 2.9: Проверка корректности считывания созданного текстового файла

2.4 Словари

При работе с данными бывает удобно записать их в формате словаря.

Предположим, что словарь должен содержать перечень всех языков программирования и года их создания, при этом при указании года выводить все

языки программирования, созданные в этом году.

При инициализации словаря можно задать конкретные типы данных для ключей и значений (рис. 2.10):

```
# Инициализация словаря:  
dict = Dict{Integer,Vector{String}}()  
  
Dict{Integer, Vector{String}}()
```

Рис. 2.10: Инициализация словаря

Можно инициировать пустой словарь, не задавая строго структуру (рис. 2.11):

```
# Инициализация словаря:  
dict2 = Dict()  
  
Dict{Any, Any}()
```

Рис. 2.11: Инициализация пустого словаря

Далее требуется заполнить словарь ключами и годами, которые содержат все языки программирования, созданные в каждом году, в качестве значений (рис. 2.12):

```
# Заполнение словаря данными:  
for i = 1:size(P,1)  
    year,lang = P[i,:]  
    if year in keys(dict)  
        dict[year] = push!(dict[year],lang)  
    else  
        dict[year] = [lang]  
    end  
end
```

Рис. 2.12: Заполнение словаря данными

В результате при вызове словаря можно, выбрав любой год, узнать, какие языки программирования были созданы в этом году (рис. 2.13):

```
# Пример определения в словаре языков программирования, созданных в 2003 году  
dict[2003]
```

```
2-element Vector{String}:  
 "Groovy"  
 "Scala"
```

Рис. 2.13: Пример работы словаря

2.5 DataFrames

Работа с данными, записанными в структуре DataFrame, позволяет использовать индексацию и получить доступ к столбцам по заданному имени заголовка или по индексу столбца.

На примере с данными о языках программирования и годах их создания зададим структуру DataFrame (рис. 2.14):

<pre># Подгружаем пакет DataFrames: using DataFrames # Задаём переменную со структурой DataFrame: df = DataFrame(year = P[:,1], language = P[:,2])</pre>			↑ ↓
73×2 DataFrame			48 r
Row	year	language	
	Int64	String31	
1	1951	Regional Assembly Language	
2	1952	Autocode	
3	1954	IPL	
4	1955	FLOW-MATIC	
5	1957	FORTRAN	
6	1957	COMTRAN	
7	1958	LISP	
8	1958	ALGOL 58	

Рис. 2.14: Пример создания структуры DataFrame

Если требуется получить доступ к столбцам по имени заголовка, то необходимо добавить к имени заголовка двоеточие. В Julia это означает, что имена заголовков обрабатываются как символы. Также следует иметь в виду, что вызов `df[1]` эквивалентен вызову `df[:year]`. Пакет DataFrames предоставляет возможность с помощью `description` получить основные статистические сведения о каждом столбце во фрейме данных(рис. 2.15).


```
# Вывод всех значения столбца year:
df[, :year]
```

```
73-element Vector{Int64}:
 1951
 1952
 1954
 1955
 1957
 1957
 1958
 1958
 1959
 1959
 1959
 1962
 1962
  ⋮
2003
2005
2006
2007
2009
2010
2011
2011
2011
2011
2012
2014
```

```
# Получение статистических сведений о фрейме:
describe(df)
```

2×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	year	1982.99	1951	1986.0	2014	0	Int64

Рис. 2.15: Пример создания структуры DataFrame

2.6 RDatasets

С данными можно работать также как с наборами данных через пакет RDatasets языка R (рис. 2.16):

```
# Подгружаем пакет RDatasets:
using RDatasets
# Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")
```

150×5 DataFrame

125 rows

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
⋮	⋮	⋮	⋮	⋮	⋮
139	6.0	3.0	4.8	1.8	virginica
140	6.9	3.1	5.4	2.1	virginica
141	6.7	3.1	5.6	2.4	virginica
142	6.9	3.1	5.1	2.3	virginica

Рис. 2.16: Работа с пакетом RDatasets

Пакет RDatasets также предоставляет возможность с помощью `description` получить основные статистические сведения о каждом столбце в наборе данных (рис. 2.17):

```
# Определения типа переменной:  
typeof(iris)
```

DataFrame

```
describe(iris)
```

5×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

Рис. 2.17: Получение основных статических сведений о каждом столбце в наборе данных

2.7 Работа с переменными отсутствующего типа (Missing Values)

Пакет DataFrames позволяет использовать так называемый «отсутствующий» тип (рис. 2.18). В операции сложения числа и переменной с отсутствующим типом значение также будет иметь отсутствующий тип.

```
# Отсутствующий тип:  
a = missing  
typeof(a)
```

Missing

```
# Пример операции с переменной отсутствующего типа:  
a + 1
```

missing

Рис. 2.18: Использование “отсутствующего” типа. Операция сложения числа и переменной с отсутствующим типом

Приведём пример работы с данными, среди которых есть данные с отсутствующим типом.

Предположим есть перечень продуктов, для которых заданы калории. В массиве значений калорий есть значение с отсутствующим типом (рис. 2.19):

```
: # Определение перечня продуктов:  
foods = ["apple", "cucumber", "tomato", "banana"]  
# Определение калорий:  
calories = [missing, 47, 22, 105]
```

```
: 4-element Vector{Union{Missing, Int64}}:  
 missing  
    47  
    22  
   105
```

```
: # Определение типа переменной:  
typeof(calories)
```

```
: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рис. 2.19: Пример работы с данными, среди которых есть данные с отсутствующим типом

При попытке получить среднее значение калорий, ничего не получится из-за наличия переменной с отсутствующим типом.

Для решения этой проблемы необходимо игнорировать отсутствующий тип (рис. 2.20):

```
# Подключаем пакет Statistics:
using Statistics
# Определение среднего значения:
mean(calories)
```

```
missing
```

```
# Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))
```

```
58.0
```

Рис. 2.20: Игнорирование отсутствующего типа

Далее показано, как можно сформировать таблицы данных и объединить их в один фрейм (рис. 2.21):

```
# Задание сведений о ценах:
prices = [0.85,1.6,0.8,0.6]
# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods,calories=calories)
# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods,price=prices)
# Объединение данных о калориях и ценах:
DF = innerjoin(dataframe_calories,dataframe_prices,on=:item)
```

4×3 DataFrame

Row	item	calories	price
	String	Int64?	Float64
1	apple	missing	0.85
2	cucumber	47	1.6
3	tomato	22	0.8
4	banana	105	0.6

Рис. 2.21: Формирование таблиц данных и их объединение в один фрейм

2.8 FileIO

В Julia можно работать с так называемыми «сырыми» данными, используя пакет FileIO.

Попробуем посмотреть, как Julia работает с изображениями.

Подключим соответствующий пакет. Загрузим изображение (в данном случае логотип Julia)(рис. 2.22):

```
# Подключаем пакет FileIO:
using FileIO

# Подключаем пакет ImageIO:
import Pkg
Pkg.add("ImageIO")

Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`

# Загрузка изображения:
X1 = load("julialogo.png")
```

Рис. 2.22: Подключение пакетов. Загрузка изображения

2.9 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

Задача кластеризации данных заключается в формировании однородной группы упорядоченных по какому-то признаку данных.

Метод k-средних позволяет минимизировать суммарное квадратичное отклонение точек кластеров от центров этих кластеров.

Рассмотрим задачу кластеризации данных на примере данных о недвижимости. Файл с данными houses.csv содержит список транзакций с недвижимо-

стью в районе Сакраменто, о которых было сообщено в течение определённого числа дней.

Загрузим данные (рис. 2.23):

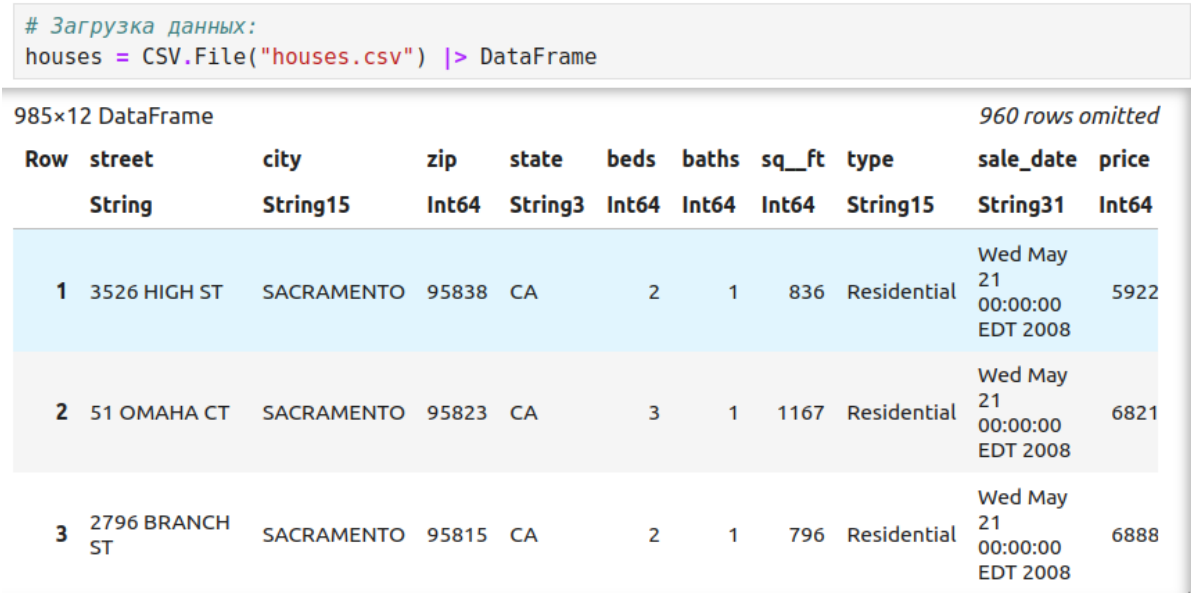


Рис. 2.23: Загрузка данных

Построим график цен на недвижимость в зависимости от площади (рис. 2.24):

```
# Построение графика:
using Plots
plot(size=(500,500),leg=false)
x = houses[:, :sq_ft]
y = houses[:, :price]
scatter(x,y,markersize=3)
```

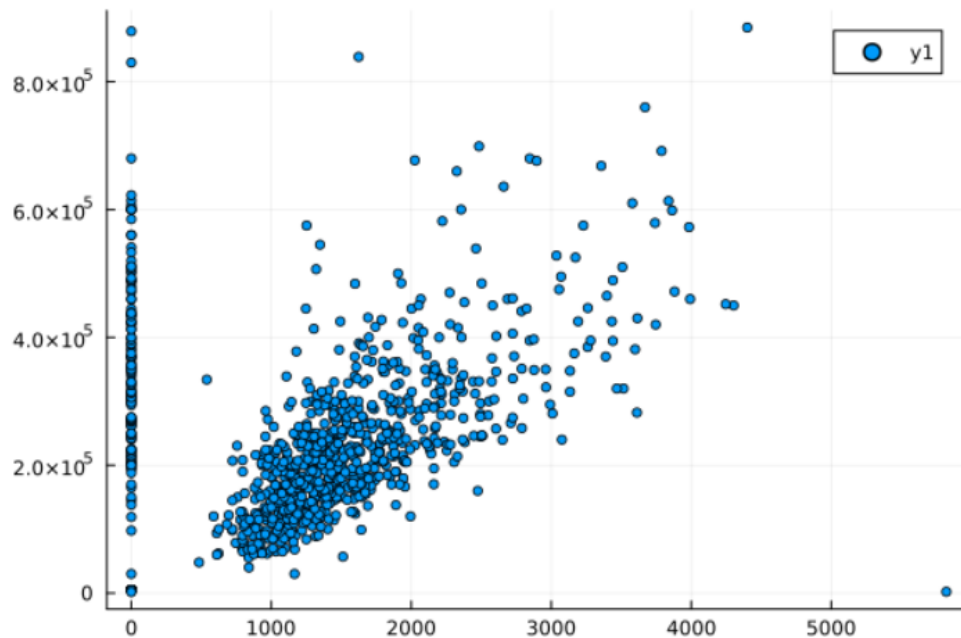


Рис. 2.24: Построение графика цен на недвижимость в зависимости от площади

Для того чтобы избавиться от “артефактов”, можно отфильтровать и исключить такие значения, получить более корректный график цен(рис. 2.25):


```
# Фильтрация данных по заданному условию:
filter_houses = houses[houses[:,sq_ft]>0,:]
# Построение графика:
x = filter_houses[:,sq_ft]
y = filter_houses[:,price]
scatter(x,y)
```

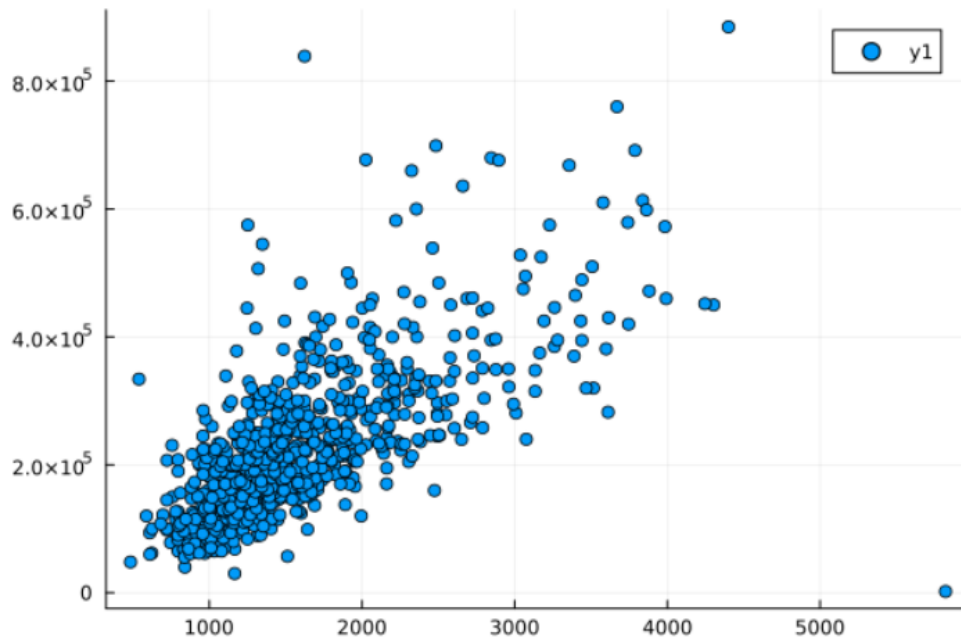


Рис. 2.25: Построение графика без “артефактов”

Используя для фильтрации значений функцию `by` пакета `DataFrames` и для вычисления среднего значения функцию `mean` пакета `Statistics`, можно посмотреть среднюю цену домов определённого типа (рис. 2.26).

```
# Подключение пакета Statistics:  
using Statistics  
# Определение средней цены для определённого типа домов:  
combine(groupby(filter_houses, :type), :price => mean)
```

3×2 DataFrame

Row	type	price_mean
	String15	Float64
1	Residential	2.34802e5
2	Condo	1.34213e5
3	Multi-Family	2.24535e5

Рис. 2.26: Среднее значение

Построим график, обозначив каждый кластер отдельным цветом (рис. 2.27 - рис. 2.28):

```

import Pkg
Pkg.add("Clustering")
using Clustering
# Добавление данных :latitude и :longitude в новый фрейм:
X = filter_houses[:, [:latitude,:longitude]]

#Преобразование DataFrame в матрицу
X = Matrix{Float64}(X) '
k = length(unique(filter_houses[:, :zip]))

# Определение кластеров k-среднего
C = kmeans(X, k)

# Формирование DataFrame с результатами кластеризации
df = DataFrame(
    cluster = C.assignments,
    city = filter_houses[:, :city],
    latitude = filter_houses[:, :latitude],
    longitude = filter_houses[:, :longitude],
    zip = filter_houses[:, :zip]
)

# Построение графиков кластеров
clusters_figure = plot(legend=false)
for i in 1:k
    clustered_houses = df[df[:, :cluster] .== i, :]
    xvals = clustered_houses[:, :latitude]
    yvals = clustered_houses[:, :longitude]
    scatter!(clusters_figure, xvals, yvals, markersize=4)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)

```

Рис. 2.27: Построение графика с кластерами разных цветов

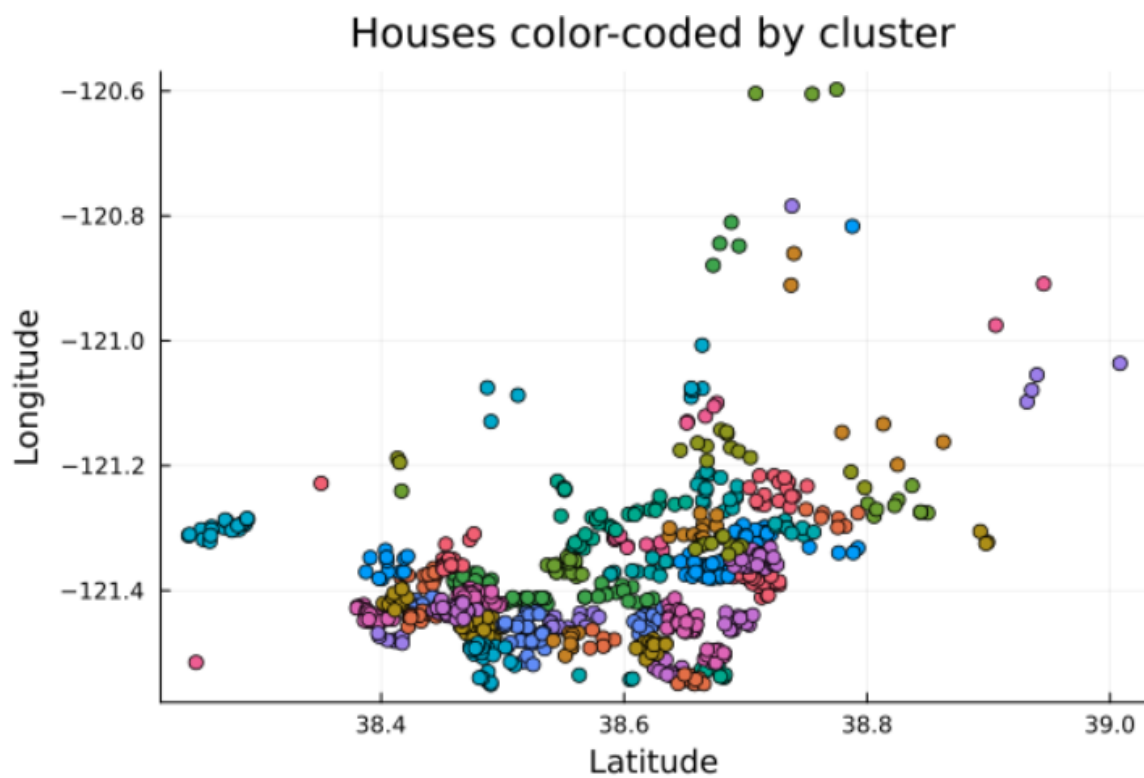


Рис. 2.28: Построение графика с кластерами разных цветов

Построим график, раскрасив кластеры по почтовому индексу (рис. 2.29):

```

unique_zips = unique(filter_houses[:,zip])
zips_figure = plot(legend = false)
for uzip in unique_zips
    subs = filter_houses[filter_houses[:,zip].==uzip,:]
    x = subs[:,latitude]
    y = subs[:,longitude]
    scatter!(zips_figure,x,y)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by zip code")
display(zips_figure)

```

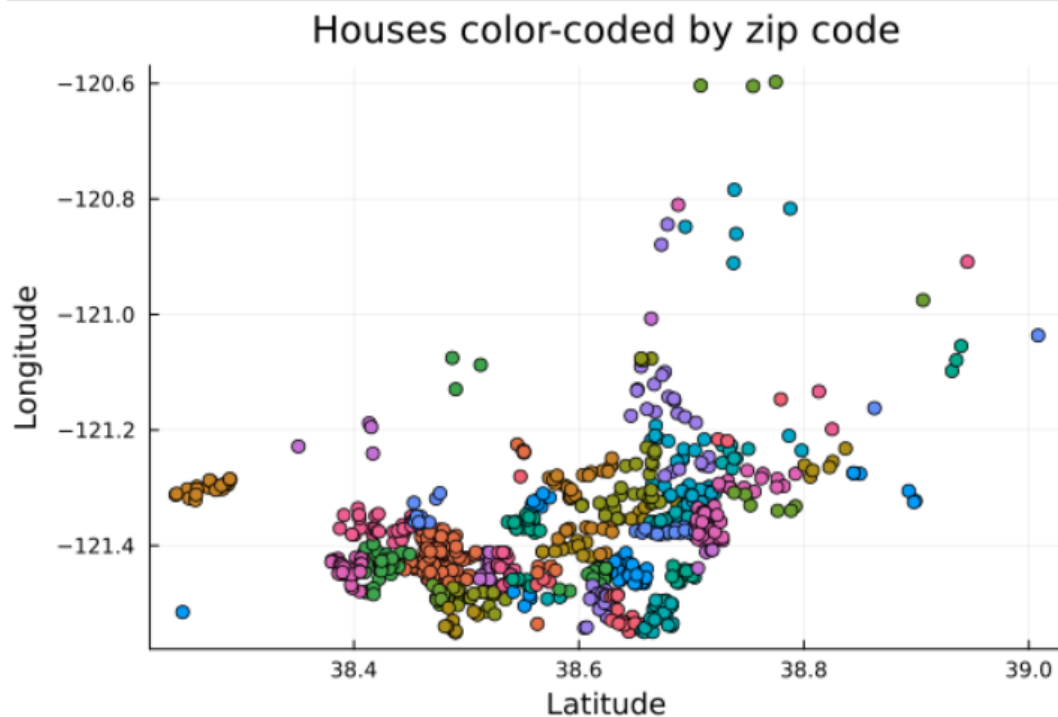


Рис. 2.29: Построение графика с кластерами разных цветов по почтовому индексу

2.10 Кластеризация данных. Метод k ближайших соседей

Отобразим на графике соседей выбранного объекта недвижимости (рис. 2.30):

```
# Все объекты недвижимости:
x = filter_houses[!,:latitude];
y = filter_houses[!,:longitude];
scatter(x,y)
# Соседи:
x = filter_houses[idxs,:latitude];
y = filter_houses[idxs,:longitude];
scatter!(x,y)
```

```
Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Proj
ml`
Manifest No packages added to or removed from `~/julia/environments/v1.12/Mani
oml`
```

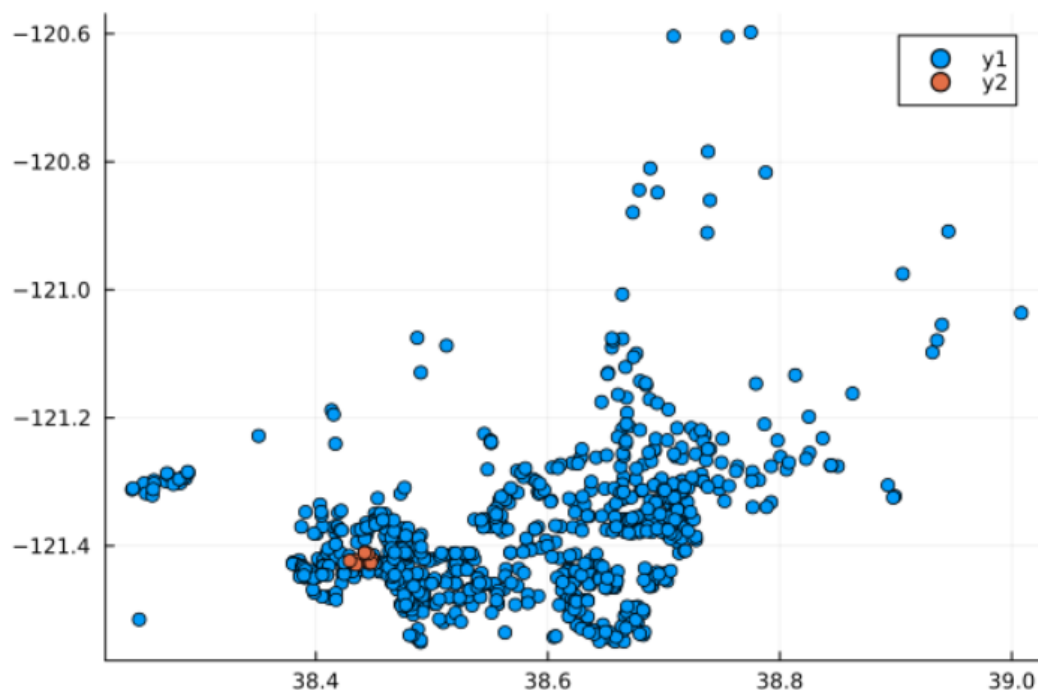


Рис. 2.30: Отображение на графике соседей выбранного объекта недвижимости

Используя индексы `idxs` и функцию `:city` для индексации в `DataFrame` `filter_houses`, можно определить районы соседних домов (рис. 2.31):

```
# Фильтрация по районам соседних домов:  
cities = filter_houses[idxs,:city]
```



```
10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:  
"SACRAMENTO"  
"ELK GROVE"  
"SACRAMENTO"  
"SACRAMENTO"  
"SACRAMENTO"  
"SACRAMENTO"  
"ELK GROVE"  
"ELK GROVE"  
"ELK GROVE"  
"ELK GROVE"
```

Рис. 2.31: Определение районов соседних домов

2.11 Обработка данных. Метод главных компонент

Метод главных компонент (Principal Components Analysis, PCA) позволяет уменьшить размерность данных, потеряв наименьшее количество полезной информации. Метод имеет широкое применение в различных областях знаний, например, при визуализации данных, компрессии изображений, в эконометрике, некоторых гуманитарных предметных областях, например, в социологии или в политологии.

На примере с данными о недвижимости попробуем уменьшить размеры данных о цене и площади из набора данных домов (рис. 2.32):

```

# Фрейм с указанием площади и цены недвижимости:
F = filter_houses[!, [:sq_ft,:price]]
y = filter_houses[!, :price]
F = Matrix(F)'
y = y'
# Приведение типов данных к распределению для PCA:
M = fit(PCA, F)
# Выделение значений главных компонентв отдельную переменную
Xr = reconstruct(M, y)
# Построение графика с выделением главных компонент
scatter(F[1,:], F[2,:])
scatter!(Xr[1,:],Xr[2,:])

```

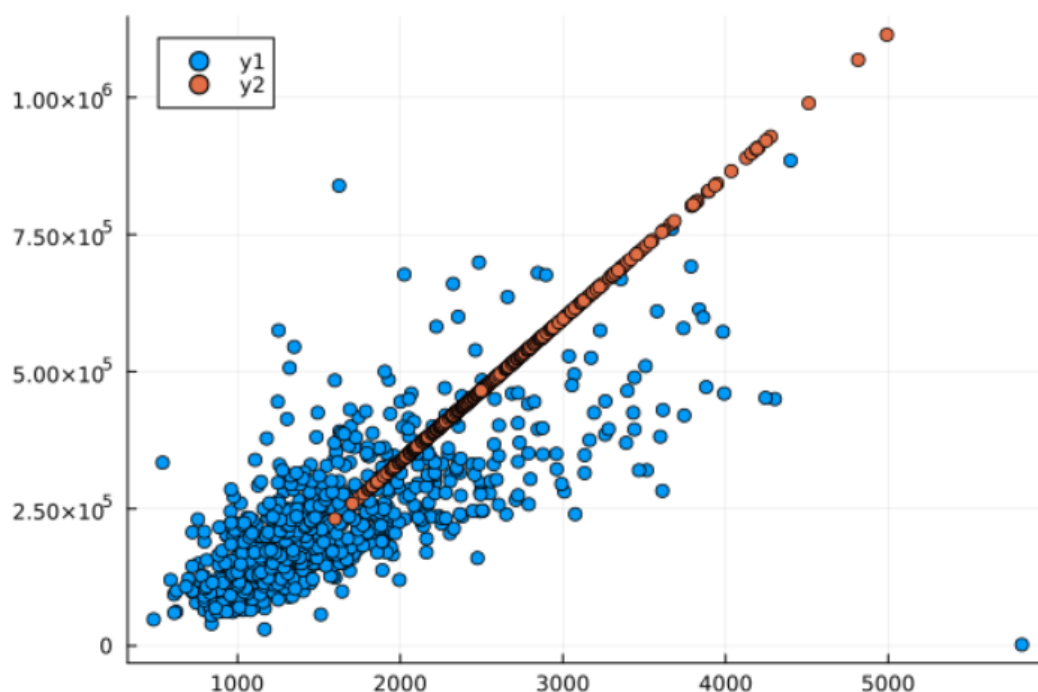


Рис. 2.32: Попытка уменьшения размера данных о цене и площади из набора данных домов

2.12 Обработка данных. Линейная регрессия

Регрессионный анализ представляет собой набор статистических методов исследования влияния одной или нескольких независимых переменных (регрессоров) на зависимую (критериальная) переменную. Терминология зависимых и независимых переменных отражает лишь математическую зависимость

переменных, а не причинноследственные отношения.

Наиболее распространённый вид регрессионного анализа — линейная регрессия, когда находят линейную функцию, которая согласно определённым математическим критериям наиболее соответствует данным.

Зададим случайный набор данных (можно использовать и полученные экспериментальным путём какие-то данные). Попробуем найти для данных лучшее соответствие (рис. 2.33):

```
xvals = repeat(1:0.5:10,inner=2)
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1
scatter(xvals,yvals,color=:black,leg=false)
```

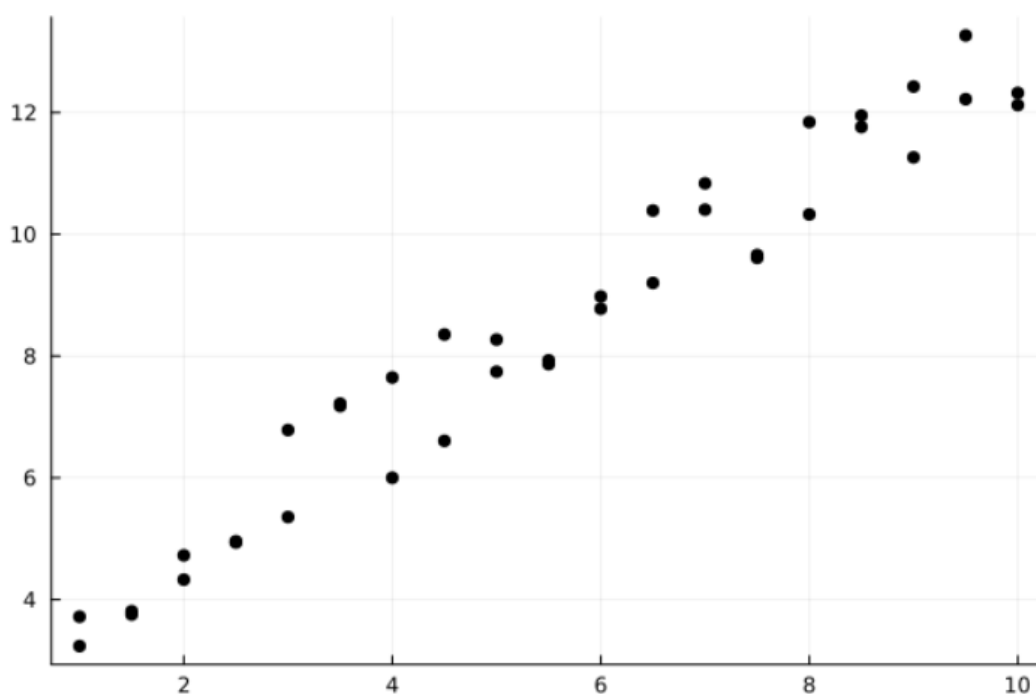


Рис. 2.33: Исходные данные

Определим функцию линейной регрессии. Применим функцию линейной регрессии для построения соответствующего графика значений (рис. 2.34):

```

function find_best_fit(xvals,yvals)
    meanx = mean(xvals)
    meany = mean(yvals)
    stdx = std(xvals)
    stdy = std(yvals)
    r = cor(xvals,yvals)
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
end

```

find_best_fit (generic function with 1 method)

```

a,b = find_best_fit(xvals,yvals)
ynew = a * xvals .+ b
plot!(xvals,ynew)

```

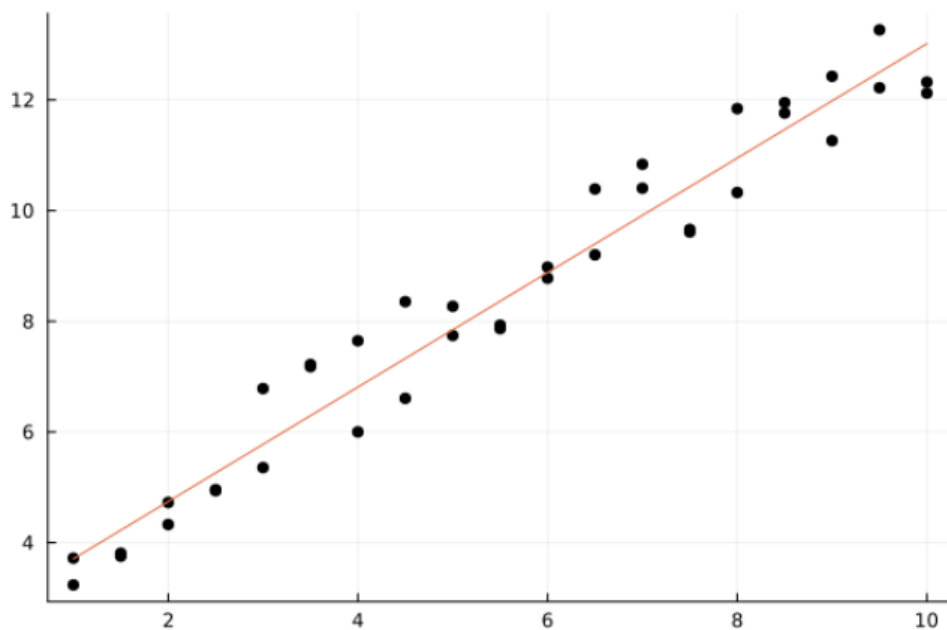


Рис. 2.34: Применение функции для построения графика

Сгенерируем большой набор данных. Определим, сколько времени потребуется, чтобы найти соответствие этим данным. Для сравнения реализуем подобный код на языке Python. Используем пакет для анализа производительности, чтобы провести сравнение (рис. 2.35 - рис. 2.36):

```

: xvals = 1:100000;
  xvals = repeat(xvals,inner=3);
  yvals = 3 .* xvals + 2*rand(length(xvals)) .* 1;
  @show size(xvals)
  @show size(yvals)

size(xvals) = (300000,)
size(yvals) = (300000,)
: (300000,)

: @time a,b = find_best_fit(xvals,yvals)

0.020382 seconds (4.38 k allocations: 230.047 KiB, 85.43% compilation time)
: (1.00000000396709656, 3.000216230582737)

: import Pkg
  Pkg.add("PyCall")
  Pkg.add("Conda")
  using PyCall
  using Conda

```

Рис. 2.35: Генерация данных. Определение времени соответствия

```

py"""
import numpy
def find_best_fit_python(xvals,yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals,yvals)[0][1]
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
"""
xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = py"find_best_fit_python"(xpy,ypy)

```

```

Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`
Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`
0.009900 seconds (100 allocations: 3.641 KiB)
(1.0000000396709676, 3.0002162304808735)

```

```

import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools
@btime a,b = py"find_best_fit_python"(xvals,yvals)
@btime a,b = find_best_fit(xvals,yvals)

```

```

Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`
4.738 ms (34 allocations: 960 bytes)
1.019 ms (1 allocation: 32 bytes)
(1.0000000396709656, 3.000216230582737)

```

Рис. 2.36: Сравнение

2.13 Самостоятельное выполнение

Загрузим

```

using RDatasets
iris = dataset("datasets", "iris")

```

Используем Clustering.jl для кластеризации на основе k-средних. Сделаем точечную диаграмму полученных кластеров. Для этого проиндексируем фрейм

данных, преобразуем его в массив и транспонируем (рис. 2.37 - рис. 2.39).

```
using RDatasets, Clustering, Plots
iris = dataset("datasets", "iris")

iris = dataset("datasets", "iris")
iris_spec = Dict{"setosa"=>1, "versicolor"=>2, "virginica"=>3}
iris[:, :5] = [iris_spec[item] for item in iris[:, :5]]

X = Matrix(iris)
k = length(unique(iris[:, :5]))
C = kmeans(X, k)
df = DataFrame(cluster = C.assignments, SepalLength = iris[:, :SepalLength],
  SepalWidth = iris[:, :SepalWidth], PetalLength = iris[:, :PetalLength],
  PetalWidth = iris[:, :PetalWidth], Species = iris[:, :Species]
)

clusters_figure = plot(legend = false)
for i = 1:k
  clustered_houses = df[df[:, :cluster].== i, :]
  xvals = clustered_houses[:, :SepalWidth]
  yvals = clustered_houses[:, :SepalLength]
  scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("SepalWidth")
ylabel!("SepalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

Рис. 2.37: Кластеризация

```
clusters_figure = plot(legend = false)
for i = 1:k
  clustered_houses = df[df[:, :cluster].== i, :]
  xvals = clustered_houses[:, :SepalWidth]
  yvals = clustered_houses[:, :SepalLength]
  scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("SepalWidth")
ylabel!("SepalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

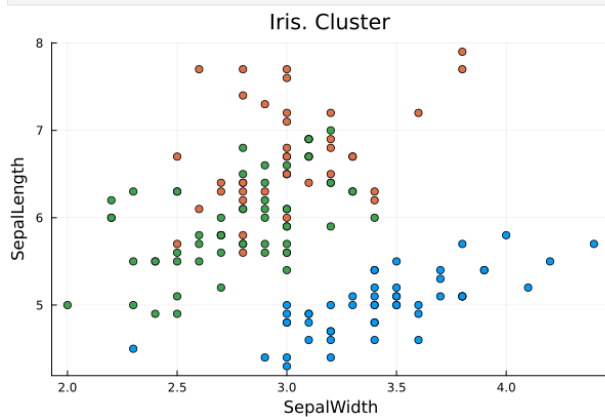


Рис. 2.38: Кластеризация

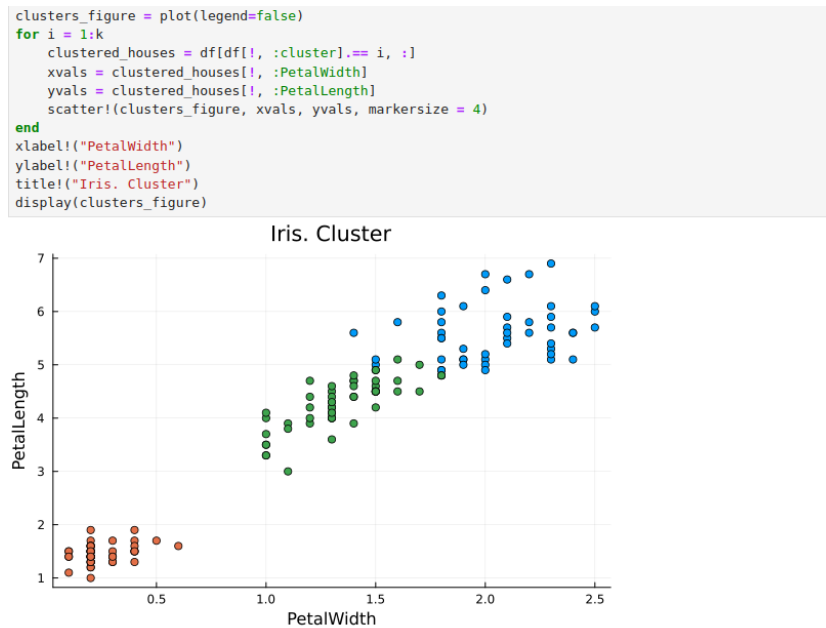


Рис. 2.39: Кластеризация

Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ `randn(N, 3)`, массив результатов в $N \times 1$, регрессионная зависимость является линейной. Найдем МНК-оценку для линейной модели (рис. 2.40 - рис. 2.41).

- Сравним свои результаты с результатами использования `llsq` из `MultivariateStats.jl`.
- Сравним свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`.

Создадим матрицу данных $X2$, которая добавляет столбец единиц в начало матрицы данных, и решим систему линейных уравнений.

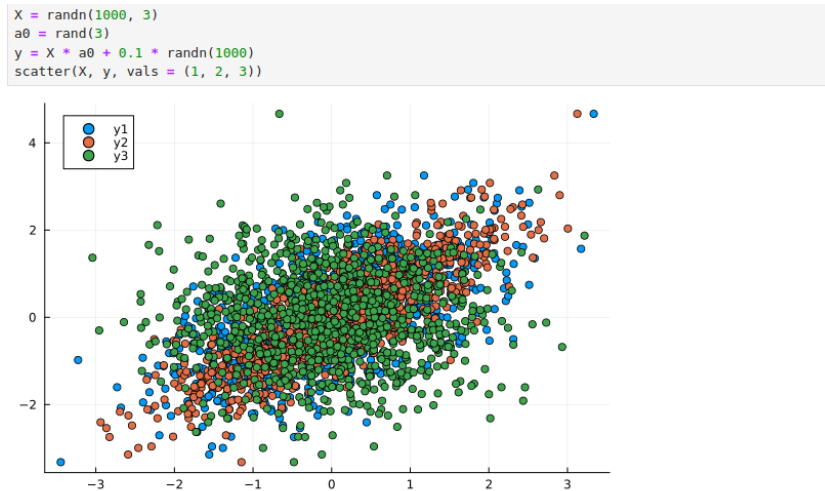


Рис. 2.40: Регрессия

```

X2 = hcat(ones(1000), X)
beta = (X2' * X2) \ (X2' * y)

4-element Vector{Float64}:
-0.00286621504429712
 0.6341176413533149
 0.8532612112348272
 0.03153307554101222

using MultivariateStats
beta_mvs = llsq(X, y)

4-element Vector{Float64}:
 0.6341176413533149
 0.8532612112348268
 0.031533075541012215
-0.0028662150442970947

Pkg.add("GLM")
using GLM
df = DataFrame(hcat(X, y), :auto)
rename!(df, ["x1", "x2", "x3", "y"])
model = lm(@formula(y ~ x1 + x2 + x3), df)
coef(model)

Resolving package versions...
Project No packages added to or removed from `~/julia/environments/v1.12/Project.toml`
Manifest No packages added to or removed from `~/julia/environments/v1.12/Manifest.toml`

4-element Vector{Float64}:
-0.0028662150442971034
 0.6341176413533149
 0.853261211234827
 0.031533075541012215

```

Рис. 2.41: Регрессия

Найдем линию регрессии, используя данные (X, y) . Построим график (X, y) , используя точечный график. Добавим линию регрессии, используя `abline!`. Добавим заголовок «График регрессии» и подпишем оси x и y (рис. 2.42).

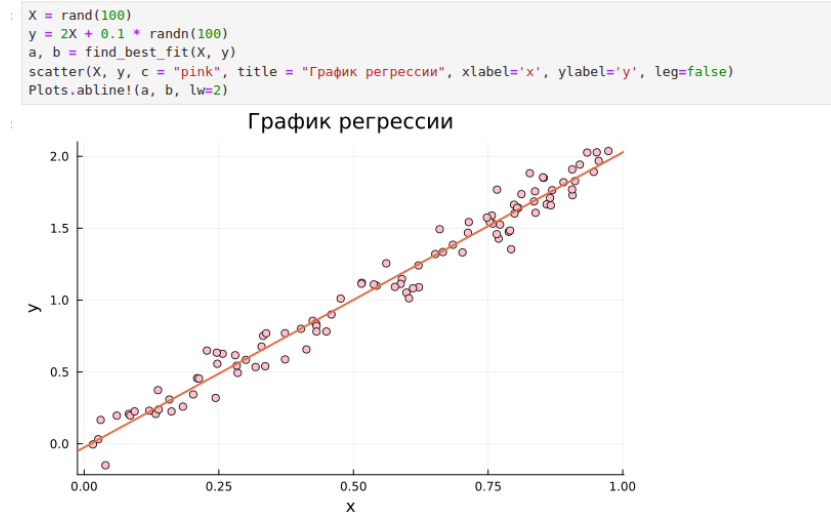


Рис. 2.42: Регрессия

Построим траекторию возможных цен на акции:

- S – начальная цена акции;
- T – длина биномиального дерева в годах;
- n – количество периодов;
- $h = Tn$ – длина одного периода;
- σ – волатильность акции;
- r – годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$;

Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуем построить траекторию курса акций(рис. 2.43 - рис. 2.44).


```

S = 100.0
T = 1.0
n = 1000
sigma = 0.3
r = 0.08
h = T / n

u = exp(r * h + sigma * sqrt(h))
d = exp(r * h - sigma * sqrt(h))
p = (exp(r * h) - d) / (u - d)

function create_path(S, r, sigma, T, n)
    path = []
    S_ = S
    push!(path, S)
    for i in 1:n
        if rand() < p
            S_ *= u
        else
            S_ *= d
        end
        push!(path, S_)
    end
    return path
end

path = create_path(S, r, sigma, T, n)
plot(path, label="Stock Price Path", xlabel="Time Step", ylabel="Price", title="Stock Price Trajectory")

```

Рис. 2.43: Модель ценообразования биномиальных опционов



Рис. 2.44: Модель ценообразования биномиальных опционов

Создадим функцию `createPath` (`S :: Float64`, `r :: Float64`, `sigma :: Float64`, `T :: Float64`, `n :: Int64`), которая создает траекторию цены акции с учетом начальных параметров. Используем `createPath`, чтобы создать 10 разных траекторий и построим их все на одном графике (рис. 2.45 - рис. 2.46).

```

function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect(0:T/n:1)
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r * h) - d) / (u - d)
    for i = 0:n
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return(X, Y)
end

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p

```

Рис. 2.45: Модель ценообразования биномиальных опционов

```

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p

```

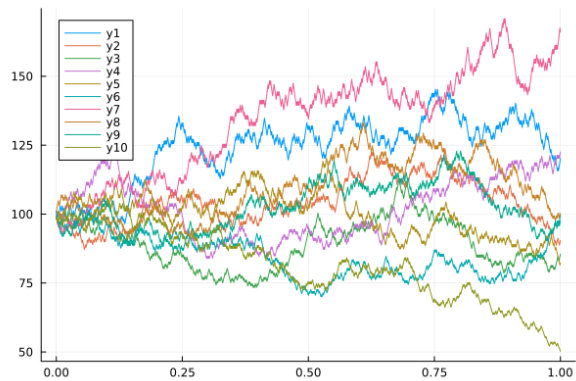


Рис. 2.46: Модель ценообразования биномиальных опционов

Распараллелим генерацию траектории. Можем использовать `Threads.@threads`, `par` и `@parallel`(рис. 2.47 - рис. 2.48).

```

function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect(0:T/n:1)
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r * h) - d) / (u - d)
    Threads.@threads for i = 0:n
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return(X, Y)
end

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p

```

Рис. 2.47: Модель ценообразования биномиальных опционов

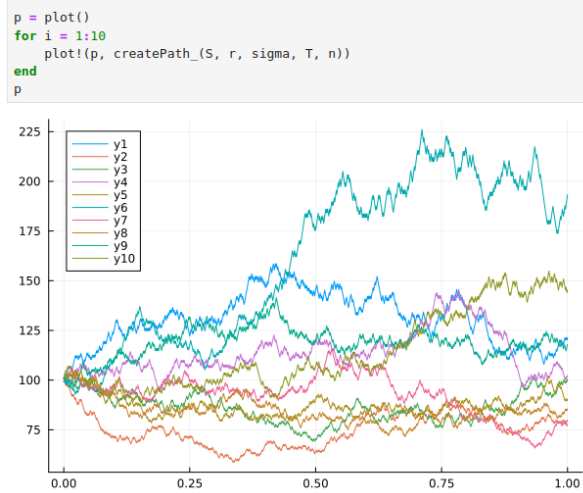


Рис. 2.48: Модель ценообразования биномиальных опционов

3 Вывод

В ходе выполнения лабораторной работы были изучены специализированные пакеты Julia для обработки данных.

Список литературы